

BalatroQt

基于 Qt6 / C++17 的卡牌肉鸽游戏《Balatro》复刻

程序设计课程大作业报告

小组编号: 125

小组成员: 肖楚童 朱韬宇 罗铭皓

技术栈: C++17 / Qt 6 Widgets / QtOpenGL / CMake + Ninja

规模: 约 3.8 万行源码, 单一桌面可执行程序

1 程序功能介绍

BalatroQt 是用 Qt6 / C++17 重新实现的卡牌肉鸽 (roguelike) 游戏《Balatro》。整个游戏被编译为一个桌面可执行程序, 使用 CMake + Ninja (MinGW) 构建, 界面基于 Qt Widgets 与 QtOpenGL。它在高度还原原版玩法的基础上, 叠加了一层以 C++ 语言概念为主题的原创内容。

1.1 核心玩法链

玩家在一局游戏中循环经历: 选择盲注 → 出牌 / 弃牌计分 → 回合结算 → 进入商店 → 迎战 Boss 盲注, 逐层推进底注 (Ante)。每次出牌由扑克牌型决定基础筹码与倍率, 最终得分为 筹码 × 倍率, 需达到目标分才能通过盲注。通关 Ante 8 后可进入无尽模式继续游玩。

1.2 内容规模

- **13 种牌型**: 从高牌、对子到同花五条, 分值集中在 `constants.h` 统一管理。
- **100 余张小丑牌 (Joker)**: 覆盖加筹码、加倍率、乘倍率、经济联动、计数器、造牌、牌型判定修正等多种机制, 并实现了蓝图 / 头脑风暴的“复制其他小丑效果”与传奇小丑。
- **三类消耗牌**: 塔罗牌、行星牌、幻灵牌, 配合增强 / 版本 / 蜡封三套卡牌修饰系统。
- **商店与经济**: 小丑、消耗牌、补充包、优惠券 (Voucher) 四类商品, 利息、赌注贴纸 (Stake) 等经济机制。
- **Boss 盲注**: 多种 Boss debuff (封禁花色、背面发牌、强制牌型等)。
- **15+ 种牌组皮肤**, 外加两套原创扩展牌组。

1.3 视觉与演示特性

- **GPU 着色器特效**: 卡面的金箔 / 全息 / 多彩 / 负片版本、卡牌溶解、火焰、漩涡、CRT 等均由 GLSL 片元着色器在离屏 FBO 中渲染为 `QPixmap`; GL 初始化失败时自动回退到 CPU 实现, 两条路径保持同步。
- **事件驱动的计分动画**: 每一笔筹码 / 倍率贡献都被记录为一个评分事件, UI 按事件顺序逐一回放成动画, 所见即所得。
- **路演演示模式**: 用一套静态状态机替换全部随机数, 使录屏与现场演示每次走相同剧本, 结果可复现。

2 各模块与类的设计细节

项目采用严格的四层架构, 依赖方向单向为 `ui` → `game` → `card` → `utils`。UI 层从不直接修改游戏状态, 只调用 `GameState` 的成员函数, 并响应其发出的信号进行刷新——这是整个工程能在万行规模下保持可维护的关键约束。

模块目录	职责
<code>src/card/</code>	纯数据值类型 + 每个实体的图形视图
<code>src/game/</code>	纯模型逻辑（除信号外不依赖 UI）
<code>src/ui/</code>	基于 <code>QGraphicsScene</code> 的视图与动画编排
<code>src/utils/</code>	全局常量、CPU / GPU 着色器渲染

2.1 card 层：数据与视图分离

- `CardData`（值类型）封装花色、点数、增强、版本、蜡封、debuff 标记、永久加筹，以及一个进程唯一的 `uid`。所有卡牌动画都按 `uid` 追踪身份而非下标：卡牌移动时保留 `uid`，只有真正生成新牌时才调用 `assignNewUid()`，从而让动画在洗牌、复制、销毁中始终锁定正确的对象。
- `Joker` 采用回调式设计：每张小丑由 `JokerType` 枚举、触发时机 `TriggerTiming`（计分逐张 / 出牌后 / 弃牌 / 回合结束 / 被动）和一个 `std::function<void(TriggerContext&)>` 效果 `lambda` 组成。新增一张小丑只需注册一个 `lambda`，主计分流程无需改动——这是“开闭原则”在本项目中最集中的体现。
- `carditem` / `jokeritem` / `consumableitem` 是对应的 `QGraphicsObject` 视图，负责悬停倾斜、拖拽、果冻形变、翻面等动画。

2.2 game 层：单一事实源

- `GameState`（约 4400 行）是全局唯一的状态源，持有手牌、小丑、金币、分数、盲注进度、消耗品、优惠券等，并通过约 16 个信号（`handChanged`、`scoreChanged`、`jokersChanged`、`shopChanged` 等）驱动界面。
- `HandEvaluator::evaluate` 负责牌型判定：排序后判断同花 / 顺子 / 按点数分组，定出牌型并给出基础筹码与倍率。其精髓在于事件驱动计分：每个贡献（计分牌筹码、增强倍率、版本乘区、小丑乘区、蜡封重触发、玻璃破碎……）都被压入一个 `ScoreEvent` 列表，而非“静默地累加一个数字”。UI 正是依靠回放这串事件来生成逐项动画。
- `Shop` 将商品按种类滚入三行槽位（主货架 / 优惠券 / 补充包）；优惠券通过 `setter` 修改商店参数（槽位数、刷新价、各类概率），并联动 `GameState` 上的扩展字段。
- `Deck` / `GameDeckType` / `BossBlind` / `Tag` / `BoosterPack` 分别管理 52 张牌池、牌组规则、Boss debuff、跳过标签奖励与补充包内容。

2.3 ui 层：信号驱动的场景

`MainWindow`（约 1.09 万行）是整个视图的总线，将每个游戏信号接到相应的场景更新，统管场景图元、动画与各覆盖层（对局、选盲注、商店、开包、牌组查看、回合结算、启动页、选项菜单）。计分动画在大量 `QPropertyAnimation` 与 `QTimer` 之间精心编排。受限于驱动问题（全屏 `QOpenGLWidget` 上叠加原生 `QDialog` 会触发黑屏重建），所有模态一律改用场景内覆盖层实现。

2.4 面向对象核心：牌组体系的继承与多态

牌组规则集中在抽象基类 `GameDeckType` 中：`name()` / `description()` 为纯虚函数，出牌次数、弃牌次数、选牌窗口等为带默认实现的虚函数。`BalatroGameDeck`（原版 15 种皮肤）、`QueueGameDeck`（队列牌组：仅队首窗口内可选、禁止排序）、`StackGameDeck`（栈牌组：出牌必须包含栈顶新牌）均为派生类重写。更换一套玩法只需注入一个不同的子类对象，主流程代码完全不变——这是本项目对“继承与多态”最直接的运用。

2.5 程序设计概念卡：把语言特性做成可玩机制

团队最具特色的设计，是将四个 C++ 概念实现为可在牌桌上打出的卡牌，且页面同步展示真实源码：

概念	实现要点
运算符重载	<code>OperatorOverload</code> 小丑：待整条计分事件流生成完毕后，统一将每个事件的“筹码 $\leftrightarrow$ 倍率”方向互换，再按 UI 回放规则重算，保证屏幕动画终值与结算分严格一致。
类模板	<code>ClassTemplate</code> 小丑：每底注被打出的第一种牌型将其“实例化”为 <code>template< 牌型 ></code> ，此后该牌型按构成张数获得乘区加成。
迭代器	<code>Iterator</code> 增强：每次打出后该牌点数 +1 ($K \rightarrow A \rightarrow 2$ 回绕)，模型逻辑与翻面动画共用同一份递推规则 <code>iterNextRank</code> 。
浅拷贝	<code>ShallowCopy</code> 塔罗：使两张牌共享一份状态。系统为每对链接保存快照，每次手牌变更时比较哪一侧被改动，再将其状态镜像给另一侧（点数 / 花色 / 增强 / 版本 / 蜡封 / debuff / 加筹全部同步）。

2.6 两个原创算法

- 占卜（胜率预览）——`simulatePlayScore`：对当前选中的手牌做一次无副作用的计分模拟，估算能打到目标分的百分比。它镜像真实计分流程但剥离事件与随机数，所有概率触发的效果均按数学期望折算（如幸运牌按 1/5 概率折算为倍率期望），并以 `mDryRun` 标志位抑制一切对游戏状态的写入。
- 最佳出牌——`findBestPlay`：用位掩码枚举所有合法出牌子集（至多 5 张），对每个子集仅对计分牌做全排列（`std::next_permutation`）——因为存在“重复触发第一张牌”等与顺序相关的小丑，卡牌顺序会影响得分。再对每种排列调用上述模拟器取最高分。算法严格“守规矩”：队列牌组只在窗口内搜索，栈顶牌与 Boss 强制牌必带上，背面朝下（玩家看不到）的牌一律排除，不作弊。

3 小组成员分工

成员	负责内容
肖楚童	核心游戏逻辑、计分事件流、版本管理（Git 工作流）
朱韬宇	界面动画、GLSL 着色器特效、路演演示模式
罗铭皓	牌组体系（继承与多态）、四张程序设计概念卡、两个原创算法

三人在工程协作上全程使用 Git，一学期累计百余次提交，并采用双分支策略：`develop` 开发线供自由迭代，`main` 主线仅接收经过审核的合并、保证始终可构建可运行。

4 项目总结与反思

4.1 收获与亮点

- 把课程概念“玩”了出来：四张概念卡让运算符重载、类模板、迭代器、浅拷贝不再是抽象语法，而成为玩法本身——这是团队最得意的设计。
- 清晰的架构换来可扩展性：单向分层 + 单一事实源 + 信号驱动，使得 100 余张小丑、十余种牌型、多种 Boss 能在不互相破坏的前提下持续叠加；回调式小丑模型让新增内容几乎不触碰主流程。
- 事件驱动计分：把“算分”与“演分”解耦，既保证了数值正确，又支撑了复杂的逐项动画，新效果只需发出事件即可自动获得动画。

4.2 遇到的困难与反思

- 开发线的波折：`develop` 分支在迭代过程中确实出现过一些 bug 与反复，尤其在浅拷贝同步、计分动画时序与高 DPI 缩放等处。得益于“主线只收审核过的合并”这一纪律，主线十余次

合并未出现事故——这让我们真切体会到分支管理与代码评审对团队项目质量的兜底作用。

- **UI 与底层驱动的耦合：**全屏 OpenGL 场景上叠加原生对话框会黑屏，迫使我们改用场景内覆盖层；这提醒我们在跨平台图形开发中要尽早验证“看似无关”的渲染假设。
- **复杂度管理：**随着小丑数量膨胀，计分阶段的触发顺序、重触发、复制效果交织，调试成本陡增。事件流与无副作用模拟器在很大程度上缓解了这一点，但也说明在设计初期就规划好扩展点的重要性。

4.3 结语

BalatroQt 在还原一款成熟商业游戏的同时，融入了对 C++ 语言概念的原创表达，并在面向对象设计、算法实现与工程协作三个维度上得到了完整的实践。它既是一份课程作业，也是一次关于“如何在规模增长中保持代码可维护”的真实演练。